Received: 1 May 2024 | Accepted: 1 May 2024

DOI: 10.1002/cav.2249

SPECIAL ISSUE PAPER

WILEY

Momentum-preserving inversion alleviation for elastic material simulation

Heejo Jeong[1] | **Seung-wook Kim**[2] | **JaeHyun Lee**[1] | **Kiwon Um**[3] | **Min Hyung Kee**[1] | **JungHyun Han**[1]

[1]Department of Computer Science and Engineering, Korea University, Seoul, South Korea

[2]Division of Computer Engineering, Hankuk University of Foreign Studies, Seoul, South Korea

[3]LTCI, Telecom Paris, IP Paris, Paris, France

Correspondence
JungHyun Han, Department of Computer Science and Engineering, Korea University, Seoul, South Korea.
Email: jhan@korea.ac.kr

Funding information
Ministry of Science and ICT, South Korea, Grant/Award Numbers: IITP-2024-2020-0-01460, IITP-2024-2020-0-01819, 2020-0-00861; Hankuk University of Foreign Studies

Abstract

This paper proposes a novel method that enhances the optimization-based elastic body solver. The proposed method tackles the *element inversion* problem, which is prevalent in the *prediction-projection* approach for numerical simulation of elastic bodies. At the prediction stage, our method alleviates inversions such that the subsequent projection solver can benefit in stability and efficiency. To prevent excessive suppression of predicted inertial motion when alleviating, we introduce a velocity decomposition method and adapt only the non-rigid motion while preserving the rigid motion, that is, linear and angular momenta. Thanks to the respected inertial motion in the prediction stage, our method produces lively motions while keeping the entire simulation more stable. The experiments demonstrate that our alleviation method successfully stabilizes the simulation and improves the efficiency particularly when large deformations hamper the solver.

KEYWORDS

elasticity, inversion, optimization, physically-based simulation

1 | INTRODUCTION

Elastic body simulation has been studied extensively in many scientific and engineering communities. In particular, it has attracted great attention in the computer graphics field, due to its variety of applications including video games, surgical simulations and virtual try-on of clothing. Such applications often entail a challenging simulation environment, where limited computational resources are available. Therefore, efficient yet stable simulation for reliable motions of elastic bodies has been a crucial goal.

In such a challenging environment, the optimization-based numerical simulation approach, for example, the projected dynamics (PD) solver,[1] has been widely used, thanks to its efficiency and stability. In this approach, the material properties such as elasticity are modeled as constraints, and the time integration is performed in two stages: In the first stage, the next physical state is initialized with the inertial motion, ignoring the constraints (i.e., prediction). Then, in the second stage, an iterative solver projects the initialized state onto the "optimal" state satisfying both the constraints and the prediction.

The nature of the two-stage algorithm, that is, *prediction-projection* scheme, however, can disturb the stability and efficiency of the solver. If the prediction stage goes wrong, the projection stage can easily get hampered, often making the solver unstable and inefficient. In practice, when the elastic body undergoes large deformation, the prediction stage often presents *element inversion* where a vertex of an element penetrates the opposite face. In this case, we can mitigate the problem by increasing the solver's iterations yet sacrificing the efficiency. On the other hand, one can alleviate the inversion at the prediction stage as proposed by Lee et al.,[2] where the inverted elements are updated to revert the inversions by replacing the predicted inertial motions with the linear motions. This method preserves the linear momentum by enforcing a linear trajectory on the inverted element. However, such enforcement drastically dissipates angular momentum.

1.1 | Contributions

This paper proposes a new inversion alleviation method that preserves both linear and angular momenta. We introduce a novel approach that first decomposes the velocities of inverted elements into rigid and non-rigid components and then selectively resolves the inversion by adjusting the non-rigid velocities such that it can respect the inertial motion as much as possible. Consequently, our method successfully produces dynamic motions of elastic bodies and leads to better stability of the solver. We demonstrate the benefit of our method in challenging examples where large deformations are continuously made. Our experiments show that the proposed method can help the projection solver result in even more stable and efficient performance for the entire time integration of the solver.

2 | RELATED WORK

2.1 | Optimization-based solvers

The implicit integration scheme is generally a good choice for stable numerical simulation and has been widely adopted for simulation of deformable bodies since the seminal studies of Terzopoulos et al.[3] and Baraff and Witkin.[4] Researchers have shown that such an implicit approach can be reformulated as optimization problems for numerical simulations.[5,6] This approach often involves two stages: *prediction*, which first predicts a physics state forward in time, and *projection*, which iteratively solves for a valid state satisfying the given physical constraints such as elasticity and inertial motion. For real-time applications, in particular, an efficient and stable simulation method is crucial. Thus approximate solutions are generally accepted when solving these optimization problems. For example, position-based dynamics (PBD) addresses the optimization problem via the iterative individual constraint projection.[7,8] Projective dynamics (PD) alternately applies the local projection and global solve with an approximate Hessian.[1,9] The projection step has been improved by many techniques such as the Chebyshev semi-iterative approach,[10] the alternating direction method of multipliers (also known as ADMM),[11,12] and the quasi-Newton method.[13]

Despite the great stability of the backward Euler scheme, such implicit integration solvers often suffer from undesired artificial damping. To address the dissipation, Dinev et al.[14] propose FEPR, which enforced energy-momentum conservation as a post-processing procedure of an arbitrary solver. Kee et al.[15] proposed an energy-momentum conserving integration method based on PD. While they tackled momentum conservation of numerical solvers, our method targets alleviating inversions aiming at momentum conservation in the alleviated state.

2.2 | Element inversion

Numerical simulation of a deformable body generally involves a discrete representation and a tetrahedra mesh is widely used. In many practical simulation scenarios, the numerical integration of the tetrahedra elements often results in element inversion, which is a physically invalid state in the real world. This element inversion can easily hamper the solver, hence it can produce undesirable results. Consequently, several studies have addressed this inversion problem. Irving et al.[16] thoroughly diagonalized the singular values of the deformation gradient to ascertain their signs. Teran et al.,[17] Gao et al.,[18] and Sin et al.[19] broadened this approach to include fully implicit integration, which allowed natural handling of inversion in the optimization solver. These methods, however, required a comprehensive analysis of the strain tensor and elastic constitutive models. Our method, on the other hand, is built upon the work of Lee et al.,[2] which introduced a novel, efficient strategy for proactively alleviating element inversion in the prediction step of the optimization-based solver. This strategy is notable for its independence from any specific elastic constitutive model or stress forces. This method prevents the element inversion by modifying the motions of the inverted elements while suppressing the rotational and non-rigid motion of the predicted inertial motion thus suffering from dissipation problems. Our method advances this strategy with a novel alleviation using velocity decomposition and least-suppression modification.

3 | OPTIMIZATION-BASED ELASTIC BODY SOLVER

We employ an optimization-based solver as our elastic body simulation method. Aiming at an efficient and stable deformable body simulation, we use PD[1] as our underlying solver and adopt the approach of Liu et al.[13] We note that the limited memory BFGS (L-BFGS)[20] used for the projection solver is computationally efficient and supports various hyperelastic energy models (e.g., the Neo-Hookean and co-rotational models[21,22]). Section 3.1 briefly reviews our underlying solver, that is, PD. Then, Section 3.2 sketches how our method alleviates the element inversion problem in the PD framework.

3.1 | Projective dynamics

The physical state of an elastic body can be represented as a set of discrete variables. Let the body consist of m finite vertices. Then, its state at time t^n can be defined as a set of positions $\mathbf{x}^n (\in \mathbb{R}^{3m})$ and velocities $\mathbf{v}^n (\in \mathbb{R}^{3m})$. Integration of the state forward in time can be approximated via the backward Euler scheme as follows:

$$\mathbf{x}^{n+1} = \mathbf{x}^n + h\mathbf{v}^{n+1} \tag{1}$$

$$\mathbf{v}^{n+1} = \mathbf{v}^n + h\mathbf{M}^{-1}(\mathbf{f}_{\text{ext}} + \mathbf{f}_{\text{int}}(\mathbf{x}^{n+1})) \tag{2}$$

where h is the time step size, $\mathbf{M}$ is the mass matrix, $\mathbf{f}_{\text{ext}}$ represents the external forces such as gravity, and $\mathbf{f}_{\text{int}}(\mathbf{x})$ represents the internal forces.

The internal forces, $\mathbf{f}_{\text{int}}(\mathbf{x})$ in Equation (2), which model material properties of the elastic body, can be derived from an elastic potential energy, W, as follows:

$$\mathbf{f}_{\text{int}}(\mathbf{x}) = -\nabla W(\mathbf{x}). \tag{3}$$

The elastic potential energy W is evaluated using the integral of an energy density function. Depending on the simulation context, different energy models can be employed for the energy density function. Our experiments focus on the tetrahedral mesh and *stable Neo-Hookean* model [22] with the Lamé parameters μ and λ. It is a popular hyperelastic material model, thanks to its superior volume-preserving feature with improved stability.

As the first step of solving for the time integration, the predicted state $\mathbf{y}$ is evaluated via Equations (1) and (2) ignoring the elasticity as follows:

$$\mathbf{y} = \mathbf{x}^n + h\mathbf{v}^n + h^2\mathbf{M}^{-1}\mathbf{f}_{\text{ext}}. \tag{4}$$

Then, taking the elasticity into account leads to the following optimization problem:

$$\text{argmin}_{\mathbf{x}^{n+1}} \underbrace{\frac{1}{2h^2}||\mathbf{x}^{n+1} - \mathbf{y}||^2_{\mathbf{M}}}_{\text{inertial term}} + W(\mathbf{x}^{n+1}). \tag{5}$$

When the elastic body deforms, the potential energy, W, defines constraints and consequently exerts a restorative force such that the deformation can return to its rest shape according to the adopted material model.

3.2 | Inversion alleviation

As the nature of the prediction-projection scheme, the predicted state (i.e., $\mathbf{y}$) affects the subsequent solving procedure. When large deformations are present, the predicted state often undergoes inversions of elements. Such inversions can cause simulation instability and slow convergence of the optimization solver.

Our method addresses the potential instability and improves the solver by alleviating the inverted elements in the prediction stage. Our alleviation algorithm preserves both linear and angular momenta such that it can support more dynamic motions of the deformable body while effectively improving the stability of the solver. We note that our method is inspired by the previous study,[2] which addresses the inversion by suppressing the velocities except the rigid linear motion.

4 | MOMENTUM-PRESERVING INVERSION ALLEVIATION

Our method targets the inverted elements arising in the prediction stage and adjusts their non-rigid motions while preserving both the linear and angular momenta. In this section, we will first present how to resolve a single element inversion and then move on to the iterative method for resolving multiple inversions.

4.1 | Single inversion

Given an inverted tetrahedral element, without loss of generality, we can enumerate its vertices using indices $i \in \{1, 2, 3, 4\}$ and associate physical state variables per vertex such as positions $\mathbf{x}_i$, velocities $\mathbf{v}_i$, external forces $\mathbf{f}_{\text{ext},i}$, and predicted positions $\mathbf{y}_i$.

Let us define the “predicted” velocity $\tilde{\mathbf{v}}_i$ as follows:

$$\tilde{\mathbf{v}}_i = \frac{\mathbf{y}_i - \mathbf{x}_i}{h} = \mathbf{v}_i + \frac{h}{m_i}\mathbf{f}_{\text{ext},i} \tag{6}$$

where m_i denotes the mass of the corresponding vertex.

The central idea of our method is to decompose the predicted velocity of the inverted element into two parts, that is, the *rigid* part, which has no influence on deformation, and the *non-rigid* part, which deforms the element. Then, only the non-rigid part is adjusted such that our alleviation process not only preserves the linear and angular momenta of the rigid motion but also keeps the non-rigid motion as close as possible to the predicted one.

4.1.1 | Velocity decomposition

Let $\tilde{\mathbf{v}}_{\text{rigid}}$ denote the *rigid velocity*, that is, the rigid part of the predicted velocity of the inverted element. It involves the linear velocity, $\tilde{\mathbf{v}}_l$, and the angular velocity, $\tilde{\boldsymbol{\omega}}$. The linear velocity, $\tilde{\mathbf{v}}_l$, can simply be computed as the average velocity of the four vertices, that is, $\tilde{\mathbf{v}}_l = \sum_i m_i\tilde{\mathbf{v}}_i / \sum_i m_i$. To obtain the angular velocity, $\tilde{\boldsymbol{\omega}}$, we first compute the center of mass as follows:

$$\mathbf{x}_c = \frac{\sum_i m_i\mathbf{x}_i}{\sum_i m_i}. \tag{7}$$

Then, we can calculate the angular momentum, $\mathbf{L}$, and the inertia tensor, $\mathbf{I}$, as follows:

$$\mathbf{L} = \sum_i m_i[\mathbf{x}_i - \mathbf{x}_c]_\times \tilde{\mathbf{v}}_i \tag{8}$$

$$\mathbf{I} = \sum_i m_i[\mathbf{x}_i - \mathbf{x}_c]_\times([\mathbf{x}_i - \mathbf{x}_c]_\times)^T \tag{9}$$

where $[\cdot]_\times$ represents the cross product matrix. Finally, we can evaluate the angular velocity, $\tilde{\boldsymbol{\omega}} = \mathbf{I}^{-1}\mathbf{L}$.

With the linear and angular velocities, we can define the rigid velocity "of each vertex" as follows:

$$\tilde{\mathbf{v}}_{\text{rigid},i} = \tilde{\mathbf{v}}_l + [\tilde{\boldsymbol{\omega}}]_\times[\mathbf{x}_i - \mathbf{x}_c]. \tag{10}$$

Let $\tilde{\mathbf{v}}_{\text{non-rigid},i}$ denote the *non-rigid velocity*, that is, the non-rigid part of the predicted velocity. It is simply defined as $\tilde{\mathbf{v}}_i - \tilde{\mathbf{v}}_{\text{rigid},i}$, where $\tilde{\mathbf{v}}_i$ is the predicted velocity given in Equation (6). With $\tilde{\mathbf{v}}_{\text{non-rigid},i}$, we will alleviate the inversion.

4.1.2 | Non-rigid motion alleviation

Our method adjusts the non-rigid motion while maintaining the rigid one such that the linear and angular momenta are preserved. Thus, the predicted velocity of each vertex, $\tilde{\mathbf{v}}_i$ given in Equation (6) is redefined as follows:

$$\hat{\mathbf{v}}_i = \tilde{\mathbf{v}}_{\text{rigid},i} + \alpha\tilde{\mathbf{v}}_{\text{non-rigid},i} \tag{11}$$

where $\alpha (\in [0, 1])$ represents the *alleviation factor*.

The goal of our alleviation method is to find a proper value for α such that the alleviated velocities can lead to an inversion-free state while respecting the original prediction as much as possible. It is worth noting that if $\alpha = 0$, it preserves only the rigid motion, and, on the other hand, if $\alpha = 1$, it keeps the original prediction without any alleviation. Consequently, we aim to find the largest α that will lead to an inversion-free state.

To find such α, we utilize the signed volume J of the tetrahedron. It is defined in terms of the predicted position of each vertex, $\hat{\mathbf{y}}_i = \mathbf{x}_i + h\hat{\mathbf{v}}_i$, as follows:

$$J = \det\big(F(\hat{\mathbf{y}}_1, \hat{\mathbf{y}}_2, \hat{\mathbf{y}}_3, \hat{\mathbf{y}}_4)\big) \tag{12}$$

where F is the deformation gradient operator and det is the determinant operator. Note that in Equation (12), α is the only unknown and J is a cubic function of α. When an inversion-free tetrahedron is inverted in the prediction stage, $J(0) > 0$ and $J(1) < 0$. Thus, we can find α by solving the cubic polynomial equation, $J(\alpha) = 0$.

In practice, however, the physical state of each simulation step does not always guarantee an inversion-free state. Hence, $J(0) < 0$ when a tetrahedron is already inverted before the prediction. In this case, we can consider the following three possibilities. Firstly, if $J(1) > 0$, it indicates the prediction resolves the inversion; thus, we just rely on the prediction without any alleviation (i.e., $\alpha = 1$). Secondly, if $J(0) < J(1) < 0$, it indicates that the prediction alleviates the inversion; thus we also rely on the prediction. Thirdly, if $J(1) < J(0) < 0$, it indicates that the prediction worsens the inversion; thus, we entirely suppress the non-rigid motion (i.e., $\alpha = 0$).

Throughout our study, it has been observed that the coefficients of the high-order terms of the cubic polynomial in Equation (12) are sufficiently small. Hence, α can be approximated using a simple linear equation as follows:

$$\alpha = \max\left(0, \min\left(1, \frac{J(0)}{J(0) - J(1)}\right)\right). \tag{13}$$

It is worth noting that all the above-mentioned possibilities are covered by the min and max operators in Equation (13). Using α, the predicted state $\hat{\mathbf{y}}$ is updated. Finally, the objective function of the optimization problem in Equation (5) is updated with $\hat{\mathbf{y}}$.

4.2 | Multiple inversions

In general, inversion alleviation might exacerbate the predicted state because a vertex is generally shared by multiple tetrahedra. Consequently, alleviating an element may invert its neighbor(s). To address the problem, we adopt a Jacobi-style iterative alleviation algorithm. For each iteration, we first independently alleviate the inverted elements in parallel, and then for each vertex, we compute the average of the updated predictions. The algorithm terminates if no inversion remains or the iteration count exceeds the predefined threshold. Additionally, we utilize the signed volume of a tetrahedron to

TABLE 1 Statistics of the experiments.

	# of elements	Time step size (ms)	Lamé μ, λ	Alleviation type, # of iter.	Solver's iter.	Total time (ms)	Alleviation overhead
Hippo (Figure 1)	8406	16.6	150, 500	Ours, 5	10	10.29	10.7%
Box (Figure 2)	4099	16.6	120, 250	Ours, max 15	10	7.72	18.2%
Box (Figure 2)	4099	16.6	120, 250	LIA, 30	10	8.54	19.8%
Cube (Figure 4)	2911	13.3	225, 500	Ours, max 15	20	7.56	9.2%
Cube (Figure 4)	2911	13.3	225, 500	LIA, 5	20	7.14	1.7%
Toy (Figure 7)	53,819	16.6	50, 250	Ours, max 10	10	90.96	12.6%
Toy (Figure 7)	53,819	16.6	50, 250	LIA, 5	10	91.06	0.7%
Pillow (Figure 9)	53,299	16.6	20, 240	Ours, max 15	10	116.71	12.6%
Pillow (Figure 9)	53,299	16.6	20, 240	LIA, 3	10	138.55	1.3%

check the severity of inversion. If the sum of absolute signed volumes of the inverted tetrahedra increases, we stop the algorithm. We compute the signed volume every five iterations.

5 | RESULTS

This section demonstrates the improved simulation results made by our method. We mainly compare our method with the previous inversion alleviation method, which reverts the inverted elements preserving their linear motions;[2] this method will be denoted as LIA that stands for Linear Inversion Alleviation. Our experiments were made with a 3.80GHz AMD Ryzen 7 5800X 8-core processor and 32GB of memory. We utilized OpenMP for parallelization. We note that our experiments with LIA used a heuristically chosen iteration count for the alleviation as in the original work[2] (e.g., 3, 5, 10, 15, or 30). Our choice was made for fair comparisons. The statistics of our experiments are presented in Table 1.

5.1 | Hippo

This example sets up a soft body of hippo and quickly pulls its rear part in a zigzag pattern, which leads to considerably large deformations. Figure 1 compares the original PD [1] without any alleviation (red in (a)) and LIA (purple in (b)) with our method (blue in (c)). Unlike the other methods suffering from unnatural motion and simulation instability, our method presents a stable and visually pleasing elastic motion of a soft material, thanks to respecting the rotational and non-rigid motions when the inversions are alleviated.

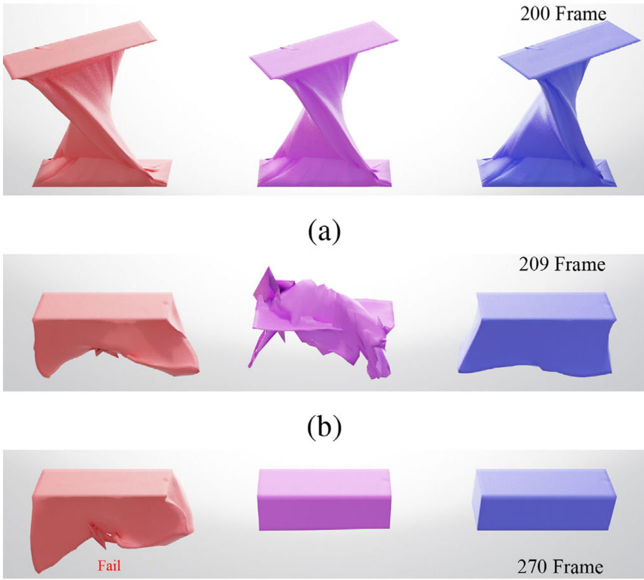
5.2 | Box

Using an elastic box, experiments are made with three methods under a more extreme inversion scenario. As shown in Figure 2, we twist the elastic box mesh upside down and release it so as to compare the recovery motions from the large deformations. Whereas PD without alleviation (in red) fails to recover and LIA (in purple) presents some visual artifacts during the recovery, our method (in blue) presents smooth recovery with visually plausible elastic motions.

We note that better alleviation of the predicted state can also help the solver's computational performance. Figure 3 presents the advantage of our method in the subsequent projection solver for a sequence of 15 frames, during the recovery

(a) (b) (c)

FIGURE 1 Hippo moving in a zigzag pattern: (a) PD without alleviation. (b) PD with LIA. (c) PD with our method.



(a)
(b)
(c)

FIGURE 2 The box undergoes compression and twisting, making the bottom face crossing over the top one: (a) The bottom face is above the top one. (b) Released states. (c) The states after a certain amount of time has passed.

motion immediately after the release. As shown in the graphs, although our method consumes slightly more computation time for the alleviation, it significantly reduces the cost for the projection solver. For these 15 frames, the entire computation time of our method is 2.66 times faster than LIA's. It is also noted that our method consumes less computational time particularly in the line search algorithm of the projection solver. During the 15 frames, our method performs 222 iterations for the line search whereas LIA performs 1114 iterations with the Wolfe conditions.

5.3 | Cube

In this experiment, we investigate dynamic motions by tracking physical quantities. As shown in Figure 4, this example sets up an elastic cube, holds an edge of the cube, and swings the edge in an elliptical motion. Figure 4a shows the initial state: PD without alleviation (in red), LIA (in purple) and our method (in blue). In Figure 4b,c, we can observe that simulation with LIA is dissipative during large deformations. In Figure 4d, we can observe that the simulation without alleviation fails.

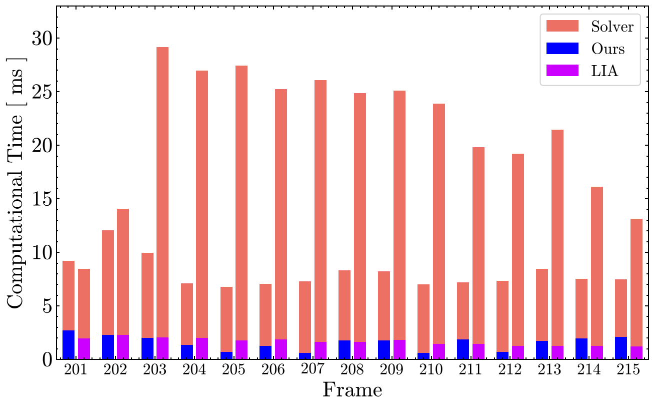


FIGURE 3 This stacked bar graph illustrates the computational time consumed with our method and LIA for each frame.

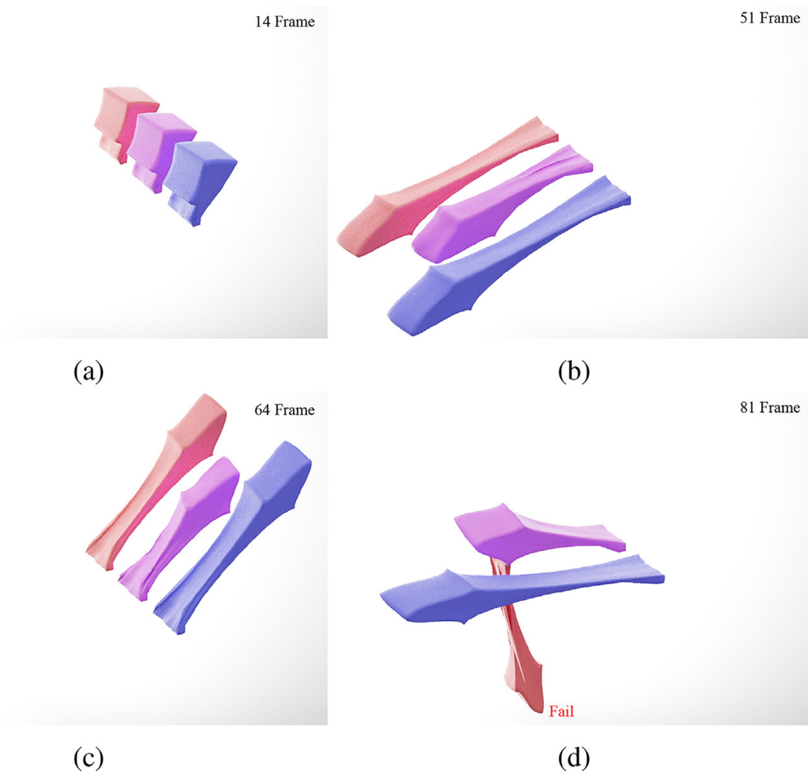


FIGURE 4 The elastic cubes swing in an elliptical motion (a) shows the initial state, (b) and (c) show that LIA (in purple) is dissipative during large deformations, (d) shows that simulation without alleviation (in red) fails.

As declared earlier, our alleviation method preserves both the linear and angular momenta. Figure 5 shows the difference in angular momentum before and after alleviation. Our method preserves the angular momentum unlike LIA.

Figure 6 presents the total energy over the simulation. We observe that our method nicely stabilizes the simulation while respecting the total energy. In contrast, the total energy blows up while simulating without alleviation. Moreover, our method presents a nicer energy evolution than LIA, which presents energy dissipation and artifacts.

As the example is configured, the continuous swinging motion leads to large deformations of the cube and causes severe inversions over time. Thus, if the inversions are ignored, the simulation easily suffers from instability although the energy evolves nicely until it blows up as shown in the graph (in orange) of Figure 6. LIA, which alleviates the inversions only for linear motion, helps avoid such instability yet hinders the nice energy evolution as shown in the graph (in purple) of Figure 6. The energy dissipation is obvious because of the potential suppression of angular motion in LIA. Moreover, we presume that the simple iterative algorithm of LIA can result in noisy states thus energy artifacts appearing as peaks in the graph.

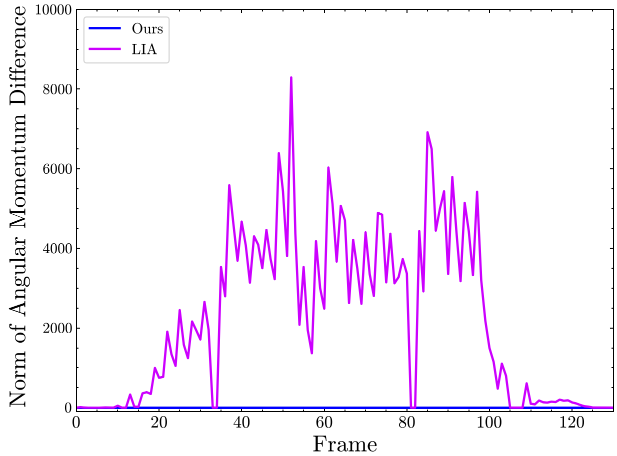


FIGURE 5 The graph illustrates the norm of the difference in angular momentum. Whereas our method preserves angular momentum, LIA does not.

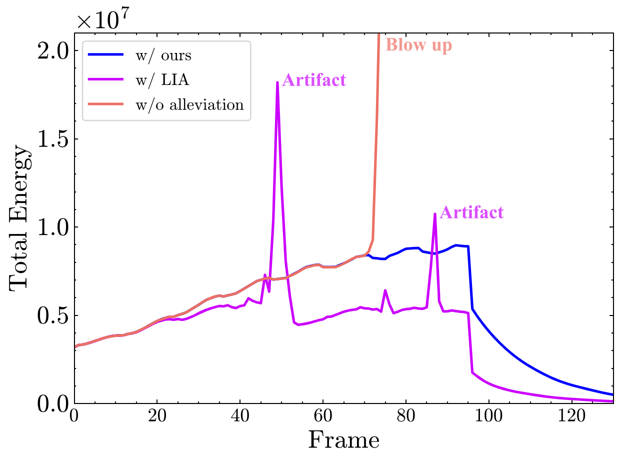


FIGURE 6 The graph depicts the total energy (kinetic energy + potential energy) over the frames.

5.4 | Toy

In this experiment, an elastic toy falls under gravity and collides with the ground. We handle the collisions using the external forces proportional to the penetration depths. Figure 7 shows visual comparisons of the three methods: original PD without alleviation (in red), LIA (in purple), and our method (in blue). Our method effectively captures the dynamics of elastic motion without failure and noisy surface artifacts, which are clearly presented in the other two methods.

Figure 8 presents the average absolute signed volumes of inverted tetrahedra after the alleviation. This measures the severity of inversion over the simulation and reflects the benefit of alleviation. We can observe that the alleviated states of LIA suffer from large fluctuation, which is presented as the noisy surface artifact in the result. Simulation without alleviation blows up. Our alleviation method successfully mitigates the fluctuation.

5.5 | Pillow

This example simulates a soft elastic pillow interacting with the wall and the ground. Figure 9 compares the three methods in a closeup view. The result of our method is superior to the others that present the simulation failure and undesirable artifacts. The teaser image presented in Figure 10 shows the overlapped still-cuts generated by our method.

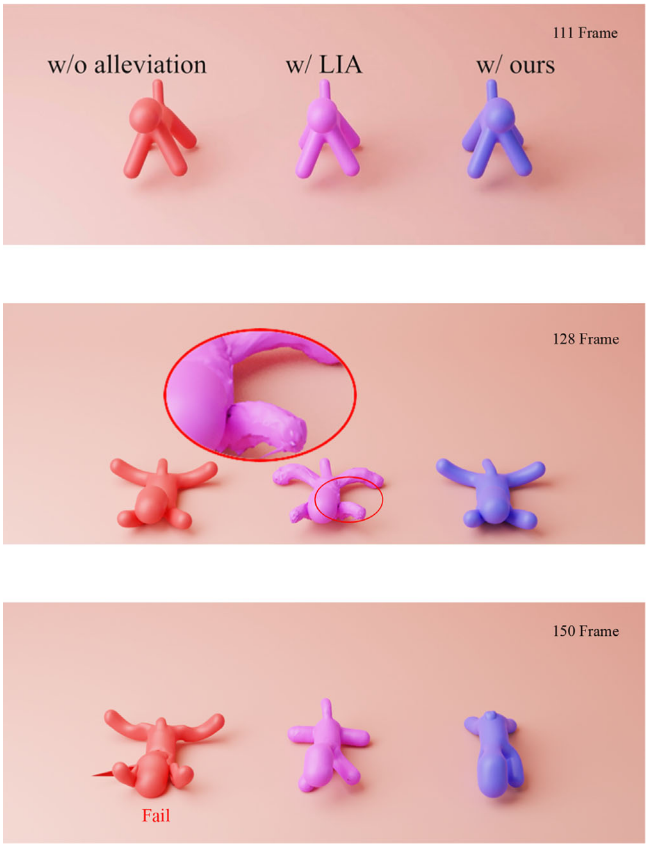


FIGURE 7 The toys undergo free fall.

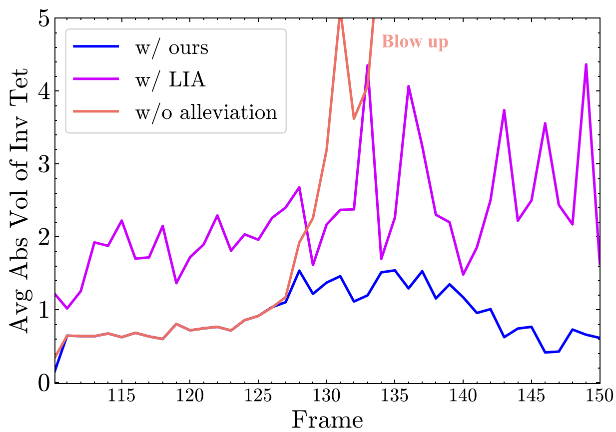


FIGURE 8 The average absolute signed volume of inverted tetrahedra after the alleviation process.

6 | LIMITATION AND FUTURE WORK

Despite the presented computational benefit in the projection solver, our method does not always outperform the other methods such as LIA due to the additional cost of our iterative algorithm. We notice that our method often performs more iterations than LIA. We assume that the algorithm aims to respect the given *erroneous* prediction as much as possible. Our method is generally beneficial when many inverted elements appear, which may hamper the projection solver.

In addition, our method does not guarantee the inversion-free state for the prediction. At the same time, more alleviation iterations do not always result in better dynamics in motion. As noted, we heuristically terminate the iteration by

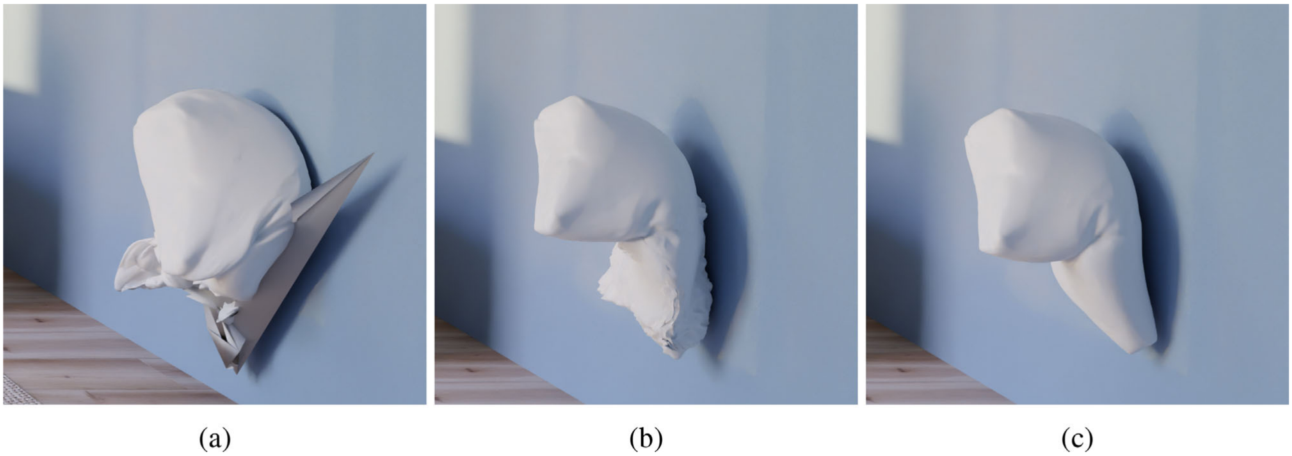


FIGURE 9 The pillow collides with the wall. (a) The result without alleviation. (b) The result with LIA. (c) The result with ours.

FIGURE 10 A soft, elastic pillow flying from left to right and colliding with the wall and ground.

checking the improvement of alleviation using the signed volumes of inverted elements. The further investigation of optimal alleviation remains our future work. Moreover, applying our method to different solvers and examining its benefit point towards avenues of future work.

7 | CONCLUSIONS

This paper introduced an advanced method that mitigates the element inversion problem of the optimization-based elastic body solver. The method focused on the prediction stage and proposed to adapt the non-rigid motion using the velocity decomposition while preserving the rigid motion (i.e., both the linear and angular momenta). The experiments demonstrated the consequent improvement in stability and the computational gain of the solver. Thanks to the "least-suppression" of the predicted inertial motion, our method successfully produced dynamic motions presenting more lively elastic behavior when it suffers from the instability without any alleviation or the artifacts with the previous alleviation method.

ACKNOWLEDGMENTS

This research was supported by the Ministry of Science and ICT, Korea, under the ICT Creative Consilience Program (IITP-2024-2020-0-01819), ITRC (Information Technology Research Center) Support Program (IITP-2024-2020-0-01460) and the grant no. 2020-0-00861. This research was also supported by Hankuk University of Foreign Studies Research Fund (of 2024).

DATA AVAILABILITY STATEMENT

Data sharing not applicable to this article as no datasets were generated or analyzed during the current study.

ORCID

JungHyun Han https://orcid.org/0000-0002-1591-3502

REFERENCES

1. Bouaziz S, Martin S, Liu T, Kavan L, Pauly M. Projective dynamics: fusing constraint projections for fast simulation. ACM Trans Graph. 2014;33(4):1–11.
2. Lee JH, Kim S-w, Um K, Kee MH, Han JH. Inversion alleviation for stable elastic body simulation. Comput Animat Virtual Worlds. 2023;34(3-4):e2183.
3. Terzopoulos D, Platt J, Barr A, Fleischer K. Elastically deformable models. Proceedings of the 14th annual conference on computer graphics and interactive techniques, pages 205–214. 1987.
4. Baraff D. Fast contact force computation for nonpenetrating rigid bodies. Proceedings of the 21st annual conference on computer graphics and interactive techniques, pages 23–34. 1994.
5. Gast TF, Schroeder C, Stomakhin A, Jiang C, Teran JM. Optimization integrator for large time steps. IEEE Trans Vis Comput Graph. 2015;21(10):1103–15.
6. Martin S, Thomaszewski B, Grinspun E, Gross M. Example-based elastic materials. ACM SIGGRAPH 2011 papers, pages 1–8. 2011.
7. Macklin M, Müller M. Position based fluids. ACM Trans Graph. 2013;32(4):1–12.
8. Macklin M, Müller M, Chentanez N. Xpbd: position-based simulation of compliant constrained dynamics. Proceedings of the 9th international conference on motion in games, pages 49–54. 2016.
9. Liu T, Bargteil AW, O'Brien JF, Kavan L. Fast simulation of mass-spring systems. ACM Trans Graph. 2013;32(6):1–7.
10. Wang H. A chebyshev semi-iterative approach for accelerating projective and position-based dynamics. ACM Trans Graph. 2015;34(6):1–9.
11. Narain R, Overby M, Brown GE. ADMM ⊇ projective dynamics: fast simulation of general constitutive models. Symposium on computer animation, pages 21–28. 2016.
12. Overby M, Brown GE, Li J, Narain R. ADMM ⊇ projective dynamics: fast simulation of hyperelastic models with dynamic constraints. IEEE Trans Vis Comput Graph. 2017;23(10):2222–34.
13. Liu T, Bouaziz S, Kavan L. Quasi-newton methods for real-time simulation of hyperelastic materials. ACM Trans Graph. 2017;36(3):1–16.
14. Dinev D, Liu T, Li J, Thomaszewski B, Kavan L. Fepr: fast energy projection for real-time simulation of deformable objects. ACM Trans Graph. 2018;37(4):1–12.
15. Kee MH, Um K, Jeong W, Han J. Constrained projective dynamics: real-time simulation of deformable objects with energy-momentum conservation. ACM Trans Graph. 2021;40(4):1–12.
16. Irving G, Teran J, Fedkiw R. Invertible finite elements for robust simulation of large deformation. Proceedings of the 2004 ACM SIGGRAPH/Eurographics symposium on computer animation, pages 131–140. 2004.
17. Teran J, Sifakis E, Irving G, Fedkiw R. Robust quasistatic finite elements and flesh simulation. Proceedings of the 2005 ACM SIGGRAPH/Eurographics symposium on computer animation, pages 181–190. 2005.
18. Gao Z, Kim T, James DL, Desai JP. Semi-automated soft-tissue acquisition and modeling for surgical simulation. 2009 IEEE international conference on automation science and engineering, IEEE, pages 268–273. 2009.
19. Sin F, Zhu Y, Li Y, Schroeder D, Barbic J. Invertible isotropic hyperelasticity using svd gradients. ACM SIGGRAPH/Eurographics symposium on computer animation (posters), volume 2. 2011.
20. Liu DC, Nocedal J. On the limited memory bfgs method for large scale optimization. Math Program. 1989;45(1):503–28.
21. Bonet J, Wood RD. Nonlinear continuum mechanics for finite element analysis. Cambridge: Cambridge University Press; 2008.
22. Smith B, De Goes F, Kim T. Stable neo-hookean flesh simulation. ACM Trans Graph. 2018;37(2):1–15.

AUTHOR BIOGRAPHIES

Heejo Jeong is a Master's student at Korea University, where he obtained his BS degree in Civil, Environmental and Architectural Engineering with an interdisciplinary major in Artificial Intelligence in 2023. His research interests focus on physics-based animation in computer graphics.

Seung-wook Kim is an assistant professor of Hankuk University of Foreign Studies. He worked as a postdoctoral researcher at Korea University where he obtained his PhD degree in Computer Science at 2021. His research interests focus on physics animation in computer graphics.

JaeHyun Lee is a Master's student at Korea University, where he obtained his BS degree in Mechanical Engineering and Computer Science in 2021. His research interests focus on physics-based simulations of natural phenomena, especially in deformable bodies.

Kiwon Um is an assistant professor of the Information Processing and Communications Laboratory (LTCI) in Telecom Paris, Institut Polytechnique de Paris. Before joining Telecom Paris, he worked as a postdoctoral researcher at the Technical University of Munich. He received his PhD, MSc, and BEng degrees at Korea University. His research interests include physics-based simulations of natural phenomena and deep learning technologies for such simulations.

Min Hyung Kee is a PhD at Korea university where he obtained his Bachelor's degree in computer science in 2018. His research interests focus on physics-based simulation, large-scale simulation and differentiable-physics simulation.

JungHyun Han is a professor in the Department of Computer Science and Engineering at Korea University, where he directs the Superintelligence Research Center. Prior to joining Korea University, he worked at the School of Information and Communications Engineering of Sungkyunkwan University in Korea, and also at the Manufacturing Systems Integration Division of the US Department of Commerce National Institute of Standards and Technology. He received a BS degree in Computer Engineering from Seoul National University, an MS degree in Computer Science from the University of Cincinnati, and a PhD degree in Computer Science from the University of Southern California.

SUPPORTING INFORMATION

Additional supporting information can be found online in the Supporting Information section at the end of this article.

How to cite this article: Jeong H, Kim S, Lee JH, Um K, Kee MH, Han JH. Momentum-preserving inversion alleviation for elastic material simulation. Comput Anim Virtual Worlds. 2024;35(3):e2249. https://doi.org/10.1002/cav.2249